

EXPERIMENT – 2 (Question 2)

Name: K ANSHU

Roll No: 2520030440

Section: 8

Title

Implementation of an Interactive Shell Loop in Linux using C to Read User Input and Handle Exit Conditions.

Aim

To create a simple interactive shell that displays a prompt, reads keyboard input, supports multi-character commands, manages the input buffer, handles the Enter key and exit condition, and tests user interaction.

Theory / Algorithm / Code

The program repeatedly displays a shell prompt, reads user input using `fgets()`, removes the newline, checks for the exit command, ignores empty input, and prints the entered command.

Procedure

1. `cd ~/OS_Week2`



A terminal window screenshot with a black background. The prompt is green text: `k_anshu@DESKTOP-IM20BAQ: ~$`. The command `pwd` has been entered, and the output `/home/k_anshu` is displayed below it.

2. `nano src/shell_loop.c`

```

k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ cat src/shell_loop.c
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 256

int main()
{
    char input[MAX_INPUT];
    while (1)
    {
        printf("MyShell> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting Shell...\n");
            break;
        }

        if (strlen(input) == 0)
            continue;

        printf("You entered: %s\n", input);
    }

    return 0;
}

```

3. Update Makefile

```

k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ cat Makefile
all:
    gcc src/fork_demo.c -o bin/fork_demo
    gcc src/shell_loop.c -o bin/shell_loop

clean:
    rm -f bin/fork_demo
    rm -f bin/shell_loop
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$

```

4. Make

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ make
gcc src/fork_demo.c -o bin/fork_demo
gcc src/shell_loop.c -o bin/shell_loop
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ |
```

5. ./bin/shell_loop

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ ./bin/shell_loop
MyShell> |
```

C Program

```
#include <stdio.h>
#include <string.h>
#define MAX_INPUT 256

int main() {
    char input[MAX_INPUT];
    while(1) {
        printf("MyShell> ");
        if(fgets(input, sizeof(input), stdin) == NULL) break;
        input[strcspn(input, "\n")] = '\0';
        if(strcmp(input, "exit") == 0) {printf("Exiting Shell...\n"); break;}
        if(strlen(input) == 0) continue;
        printf("You entered: %s\n", input);
    }
    return 0;
}
```

Sample Execution

hello

Operating Systems

OS Lab Week 2

exit

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ ./bin/shell_loop.c
MyShell> hello
You entered: hello
MyShell> Operating Systems
You entered: Operating Systems
MyShell> OS Lab Week 2
You entered: OS Lab Week 2
MyShell> operating
You entered: operating
MyShell> exit
Exiting Shell...
```

Git Commands

git status

git add .

git commit -m "Week 2 OS Experiment 2 completed"

git log --oneline

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ git log --oneline
1680115 (HEAD -> master) Week 2 OS Experiment 2 completed
a7a972c Week 2 OS Experiment 1 completed
```

Result

The interactive shell was successfully implemented and tested with keyboard input, empty input, multi-character commands, and exit handling.